Computer-controlled sinusoidal drive mechanism

Eric Ayars and Nicholas Nelson

*California State University Chico**

(Dated: May 27, 2026)

Abstract

We describe an inexpensive computer-controlled sinusoidal drive mechanism for dynamics carts (and other applications) with applications for resonance, normal-modes, and chaos experiments. The driver arm moves in a non-sinusoidal motion such that the cosine error inherent to its design *corrects* the motion, making the output a more nearly single-frequency wave than commercially-available drivers. In addition, the design facilitates easy access to the drive position at any point in time, allowing for the collection of phase information and synchronization of data to drive position.

I. INTRODUCTION

Dynamics carts and tracks are in such common use in introductory physics courses that a list of applications is unnecessary for readers of this journal. They are less commonly used in the advanced undergraduate mechanics course, however. Recently, one of the authors wanted to use dynamics carts in their upper-division mechanics course to show resonance response curves and normal modes, and needed a robust sinusoidal drive mechanism.

II. EXISTING SOLUTIONS

There are several commercial options available for this; none of them are ideal. One option is a "speaker" type driver, such as the Vernier Power Amplifier Accessory Speaker.[1] This type of driver uses an AC signal to a long-throw bass speaker coil and magnet to drive oscillations. The frequency can be precisely controlled using the function generator providing the AC signal, but the amplitude of oscillation depends on the load, and the position of the driver is only approximately sinusoidal at low amplitudes and high frequencies. There are significant higher harmonics in the driver motion which can cause experimental artifacts particularly in sensitive resonance and chaos experiments. Depending on the function generator used, a trigger signal may be available to synchronize data acquisition, but at most once per period so driver position is not precisely known. Another commercially-available option is the "circular" driver, such as the PASCO ME-8750.[2] This type uses a geared DC motor with an arm attaching a string at some radius R from the axis. The string then goes through a guide, converting the angular position θ of the motor to linear position x. (See figure 1, which also shows various parameters for later calculations.) For $L \gg R$, x is approximately sinusoidal. The amplitude of the motion for this driver is independent of load, and the frequency can be adjusted by changing the speed the motor. There are multiple limitations in the circular driver: it's impossible to adjust the amplitude on the fly, precise position information is not available, precise frequency control is difficult, and x is only approximately sinusoidal when $R \ll L$. The requirement that $R \ll L$ puts inconvenient constraints on either the maximum amplitude or the overall length of the apparatus, and the "cosine error" for R not $\ll L$ introduces significant frequency components at multiples of the drive frequency. As with the "speaker" type drier, these frequency components can cause measurement artifacts.

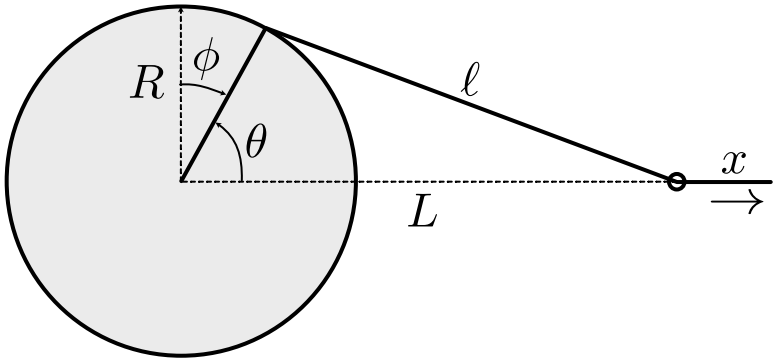

FIG. 1. Geometric layout, and variables, for both the circular driver and this servo-controlled driver

III. OUR APPROACH

We decided to use geometry identical to that of the circular driver in figure 1 but with a servomotor from a radio-control car steering system providing an oscillatory motion, instead of driving the motion in a full circle at a constant angular velocity. Controlling a servo is very easy with a microcontroller: we chose a Teensy 4.0 for its combination of low price, high speed, breadboard-friendly form factor, and floating-point math capability. Using an oscillating servomotor output allows complete on-the-fly control of both frequency and amplitude, as well as precise static positioning when desired.

The only disadvantage remaining in this approach is the existence of harmonics arising from the cosine error inherent to this geometry. Normally if one is driving a sinusoidal output with a microcontroller, the way to do it is to generate a look-up table (LUT) of output values in the microcontroller memory. The microcontroller looks up a value in the array, sets the output to that value, waits some time interval Δt, then looks up the next value and repeats the process. The frequency of the output waveform depends on the timestep Δt between LUT values. For a sinusoidal output, the look-up table would consist of values of sine (or cosine) evenly distributed over a complete period T. But we don't want the servo motion ϕ to be sinusoidal, we want x to be sinusoidal!

$$x = A\sin(\omega t) \tag{1}$$

This is equivalent to saying that

$$\ell(t) = \ell_o + A\sin(\omega t) \tag{2}$$

where

$$\ell_o \equiv \sqrt{R^2 + L^2} \tag{3}$$

The servo is centered at $\phi = 0$, and we want to find $\phi(t)$ such that $\ell(t)$ has the desired solution. Start with the law of cosines:

$$\ell^2 = R^2 + L^2 - 2RL\cos\theta \tag{4}$$

$$= R^2 + L^2 - 2RL\sin\phi \tag{5}$$

$$\ell = \left[R^2 + L^2 - rRL\sin\phi\right]^{\frac{1}{2}} = \sqrt{R^2 + L^2} + A\sin(\omega t) \tag{6}$$

A few more careful steps of algebra give us an equation for the desired motion of the servo:

$$\phi(t) = \sin^{-1}\left[\frac{1}{2RL}\left(2\ell_o A\sin(\omega t) + A^2\sin^2(\omega t)\right)\right] \tag{7}$$

Equation (7) is not particularly user-friendly, but one can check it for special cases. By geometric necessity $A < R$, so if $R \ll L$ then $\ell_o \approx L$, ϕ is small, and

$$\phi(t) \approx \frac{A}{R}\sin(\omega t) \tag{8}$$

as one would expect for this limiting-case geometry. When R is not small compared to L, one can consider the limiting case of $A \ll R$. In this case, the first term dominates and $\phi(t) \propto \sin(\omega t)$ as one would intuitively expect.

For the general case, though, the microcontroller can move the servo through the curve described by equation (7) just as easily as through a sine curve. Instead of building a LUT containing values of $A\sin(\omega t)$ at each t point, the microcontroller needs to build the LUT with values of $\phi(t)$. The values of ϕ depend on A, so the LUT must be recalculated each time the amplitude is adjusted. The Teensy microcontroller can recalculate this array of 256 LUT values in a few hundred microseconds, a negligible delay which in any case occurs only on amplitude changes.

Figure 2 shows the non-sinusoidal motion of the servo end ($R\phi$) and the resulting sinusoidal motion of the output x, as measured by a Vernier GoDirect Sensor Cart.[3]

The final construction step was to design a simple 3D-printed mount to hold the servo and the microcontroller board in place at one end of the dynamics track. See figure 3. For the fairly compact driver head we made, the improvement in the motion at maximum amplitude was an approximately 20dB decrease in the first harmonic compared to sinusoidal ϕ.

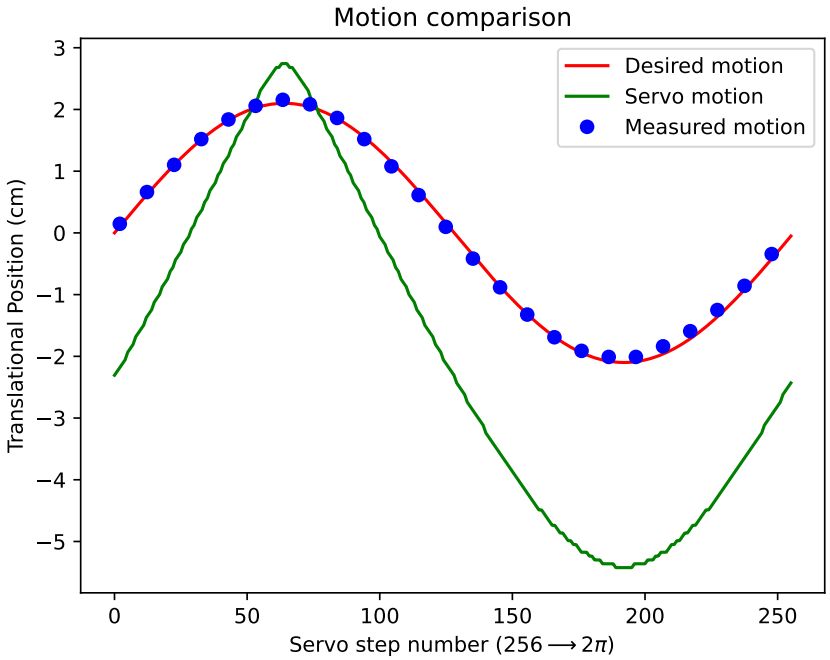

FIG. 2. A plot of the servo motion ϕ, as well as the resulting sinusoidal motion in x.

There are several advantages to having a microcontroller-based mechanical driver such as this. A microcontroller can do much more than merely drive some constant frequency and amplitude. The firmware we wrote for the microcontroller includes SCPI communications capability, so the device can be easily interfaced to LabVIEW, Python, or any other serial-capable programming language through the USB port on the Teensy 4.0 board. For the purpose of the mechanics course demonstration that sparked this project, a simple python program to control frequency and amplitude sufficed to do what was needed. Other programs can be easily created to sweep through a range of frequencies for resonance curve analysis, or a range of amplitudes for investigations of chaotic oscillators. See figure 4. In addition, it should be noted that since the microcontroller is generating the output waveform stepwise with a LUT, it is trivial to obtain exact position data for the driver at any point in time. This allows for the collection of phase information for the driver, which is not available with the commercially-available options described here. (Note the phase information in figure 4.) It also allows for the the synchronization of data to drive position, so that if for example one were using this to drive a chaotic oscillator one could collect data to generate a full set of Poincaré sections.

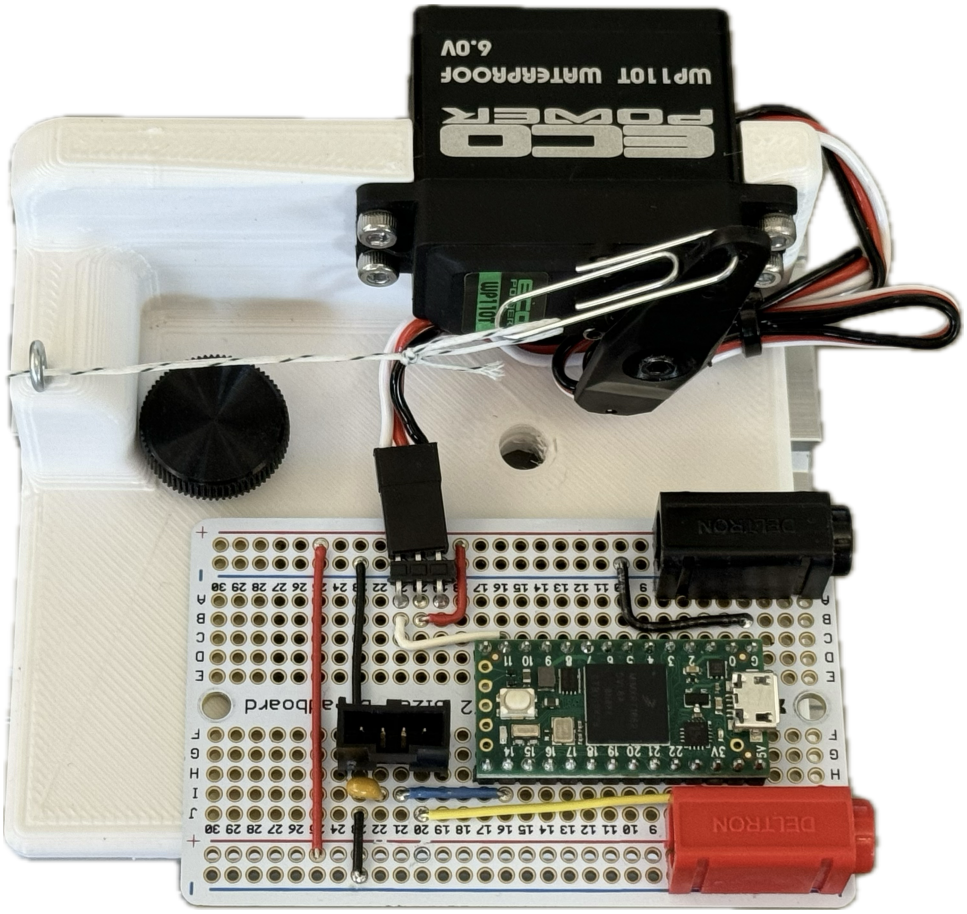

FIG. 3. A photo of the completed device, showing the circuitboard, servo, and routing of the drive string.

IV. ADDITIONAL APPLICATIONS

Although the purpose of this design is to generate a sinusoidal motion, the device is not limited to sinusoidal motion. By calculating appropriate LUT values, the microcontroller can drive any arbitrary motion of the servo arm, within the limits set by the servo speed. Applications that could be of particular interest in labs beyond the first year include multi-frequency drive output for investigations of mode-locking and other nonlinear phenomena, or driving with beats for investigations of energy transfer between modes. Modifications to the circuit and firmware could allow for experimental feedback so that the drive frequency or amplitude is adjusted in real time based on the response of the system being driven. This could be used for investigations of control theory and similar applications.

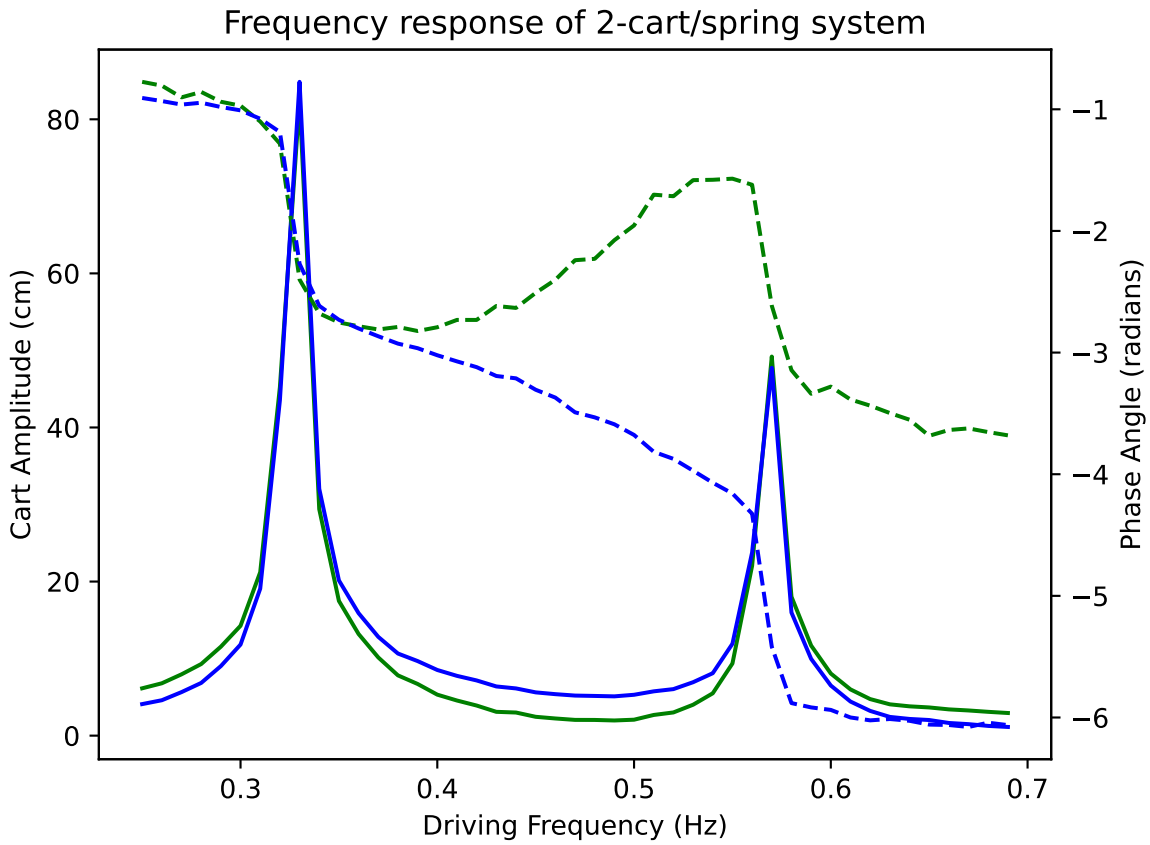

FIG. 4. Amplitude and phase response for two Vernier GoDirect Carts connected by springs, showing the normal mode resonances as well as the corresponding phase differences between the carts and the driver.

V. LIMITATIONS

The main limitation to this design of driver is the maximum frequency available. Due to the response time of the servo, the maximum frequency is around 3–5Hz, depending on the amplitude of motion. This range of frequencies works well for apparatus such as carts and Duffing oscillators and other mechanical systems, but if higher frequencies are necessary the speaker-type driver may be a better choice despite its harmonic limitations.

Another limitation is that the output becomes noticeably 'stepped' at low amplitudes. The servo is controlled by a pulse-width modulation (PWM) signal, and the resolution of the PWM signal is finite. At low amplitudes, the change in ϕ between adjacent LUT values

becomes smaller than the resolution of the PWM signal, so the servo cannot be moved to the desired position and instead jumps to one of the two closest positions it can achieve. If low amplitudes are desired, it is best to use a smaller servo arm (smaller value of R) to increase the required change in ϕ for a given change in x, but then that limits the maximum amplitude.

VI. CONCLUSIONS

By using a microcontroller to move a servo arm in a geometry for which there is cosine error, we can use the microcontroller to generate a specific non-sinusoidal motion which the cosine error then corrects to make the motion sinusoidal. The microcontroller also provides computer control of frequency, amplitude, and (if desired) static position of the driver and allows easy control of the device via Python, LabVIEW, and other programming languages. The device allows for time-synchronized position data, which opens up numerous possibilities for experiments beyond the introductory course.

VII. WANT TO BUILD ONE?

Microcontroller firmware, example Python control code, and 3D print files for this device are available on GitHub at `https://github.com/EricAyars/Cart-driver`

The Authors have no conflicts of interest to disclose.

---

* eayars@csuchico.edu

[1] Vernier Science Educaton "Power Amplifier Accessory Speaker"' <https://www.vernier.com/product/power-amplifier-accessory-speaker/>

[2] PASCO Scientific "Mechanical Oscillator Driver"' <https://www.pasco.com/products/lab-apparatus/mechanics/dynamics-accessories/mechanical-oscillator-driver>

[3] Vernier Science Education "GoDirect Sensor Cart" <https://www.vernier.com/product/go-direct-sensor-cart/>